

## Unidades o subrutinas

**Material de lectura sobre el tema:** Complementar lo presentado aquí con:

- Sebesta: Capítulo 9
- Scott: Capítulo 9

### Unidades o subrutinas

En el recorrido que venimos siguiendo, estamos andando el camino de la incorporación de la abstracción en los lenguajes de programación. Una parte de este proceso está asociado a la incorporación de las diferentes entidades con las que cuentan los lenguajes. Es por eso que en primera instancia hablamos de variables y luego de tipos. Las variables nos permitieron abstraernos de las celdas de memoria, los tipos nos permiten asociar una semántica o interpretación a lo almacenado (esta asociación nos permite establecer controles y ganar en seguridad con respecto a las operaciones que hagamos).

Estos dos pasos de abstracción son importantes pero no suficientes... es por eso que los lenguajes incorporan una nueva abstracción, en este caso se trata de abstracciones a nivel sentencia y/o expresión, poder agrupar en un nombre un comportamiento o valor más complejo. Resulta claro que este nuevo paso es fundamental ya que mientras no contemos con unidades, toda solución es monolítica. No se puede establecer ningún tipo de verificación parcial, el reúso de código se vuelve confuso, etc. Es por eso que la incorporación de mecanismos para dividir el programa en unidades (grupo de sentencias con cierta coherencia y lógica) fomenta en los lenguajes imperativos y orientados a objetos:

✧ **Eficiencia:** permiten factorizar el código (en los primeros lenguajes solo se perseguía el incremento de eficiencia principalmente de almacenamiento). Tengan en cuenta que con respecto a la administración de recursos hay un costo (en espacio y tiempo de ejecución) ineludible y que hace a un detrimento de los tiempos de ejecución y cantidad de locaciones de memoria necesarias para el mismo programa. Sin embargo este costo se amortiza de manera sobrada con la ventajas que reporta su uso.

✧ **Legibilidad:** metodología de diseño top-down (mecanismos más abstractos).

✧ **Verificación:** una unidad puede pensarse como un mapeo entre dominios de valores. Se intenta que los chequeos sean lo más seguros posibles, así evolucionaron los métodos de pasaje de parámetros.

Las **unidades** son otra de las entidades presentes en los lenguajes de programación. Como tales, también quedan definidas a través de un conjunto de atributos (cada uno de los cuales tiene su correspondiente ligadura). Los atributos que definen una unidad o subrutina son:

✧ **Nombre:** es un identificador que se asocia a la unidad. Algunos lenguajes permiten definir unidades anónimas. Las unidades anónimas se ejecutan implícitamente, habitualmente reciben el nombre de *bloque*. Por su parte, las unidades que efectivamente tienen un nombre asociado suelen llamarse *subprogramas*. A diferencia de las unidades anónimas las unidades nombradas siempre se ejecutan explícitamente.

Las unidades con nombre resultan muy interesantes para el programador ya que permiten extender al lenguaje con nuevas primitivas. Para que el nivel de abstracción se alcance de manera satisfactoria, las mismas deberían ser factibles de utilización conociendo solo su interfaz, la implementación de la misma debería ser transparente para su usuario.

✧ **Lista de parámetros:** los parámetros son la vía de comunicación de la unidad con el resto del programa. Las unidades podrán ser polimórficas si son capaces de tener algún parámetro que admita diferentes tipos.

✧ **Ambiente de referenciamiento:** este atributo establece a qué otras entidades pueden referenciar la unidad que se está definiendo.

✧ **Bloque ejecutable:** corresponde al código que implementa la unidad. Si la unidad tiene la capacidad de llamarse a sí misma, el lenguaje soporta recursividad.

✧ **Alcance:** segmento de código dentro del cual la unidad puede ser invocada (si incluye al bloque ejecutable de la misma unidad es porque el lenguaje acepta recursividad).

En los lenguajes orientados a objetos cada unidad con nombre brinda un servicio que puede ser provisto por cualquiera de las instancias de la clase en la cual se define la unidad. En este tipo de lenguajes, la clase es el mecanismo de abstracción. Un servicio puede acceder y hasta modificar el estado interno del objeto que recibe el mensaje que provoca la ejecución de ese servicio.

Mirando la lista de atributos aparecen varias cuestiones de diseño que el lenguaje debe tener en cuenta a la hora de incorporar estas entidades.

#### *Aspectos de diseño a tener en cuenta por el LP:*

- ✓ Entidades
- ✓ Ligaduras
- ✓ Reglas de alcance
- ✓ Sistemas de tipos
- ✓ Soporte para definir unidades

#### *Aspectos de diseño específicos:*

- ✓ Estructura estática
- ✓ Recursividad
- ✓ Control
- ✓ Métodos de pasaje de parámetros
- ✓ Correspondencia entre parámetros formales y actuales
- ✓ Sobrecarga y polimorfismo
- ✓ Tipos de los parámetros: datos y unidades
- ✓ Ambiente de referenciamiento de las unidades que recibe como parámetro (chequeo estático o dinámico de unidades)

Toda unidad tiene una estructura estática, definida a través de su encabezamiento y su cuerpo. El *encabezamiento* está formado por el nombre de la unidad, la lista de parámetros y el tipo del resultado (si es que tiene); mientras que el *cuerpo* está formado por las declaraciones locales y la sección ejecutable.

Si definimos el encabezamiento de manera formal, vamos a encontrar que la definición está basada en la definición de perfil de los parámetros, el protocolo y la signature.

El *perfil de los parámetros* de una unidad es el número, orden y tipo de los parámetros formales. El *protocolo* de una unidad es el perfil de los parámetros más el tipo retornado por la unidad, si es que tiene y la *signature* de una unidad está dada por el nombre de la unidad y su protocolo. La signature define la interfaz de la unidad o la parte pública de la misma.

La unidad estáticamente posee:

- Declaración.
- Definición.
- Invocación: La invocación tiene una vinculación bidireccional con la activación de la unidad y con su suspensión.

La unidad dinámicamente puede estar:

- Pasiva
- Activa – Suspendida

Su estado está determinado por las estructuras de control.

Lo ideal sería que la comunicación de una unidad esté dada solo a través de los parámetros y el resultado, pero en los paradigmas imperativo y orientado a objetos (particularmente) hay otras formas de comunicación. Las más frecuentes son:

- Ambiente global.
- Ambiente implícito.
- Parámetros y resultado.

La comunicación a través del ambiente global puede generar efectos colaterales. Recordemos que hay efecto colateral si la ejecución de una unidad modifica el ambiente de referenciamiento de la unidad llamadora. La comunicación a través del ambiente implícito puede generar comportamiento sensible a la historia.

A la hora definir unidades nos encontramos con tres escenarios posibles:

✧ **Bloques:** unidades anónimas.

✧ Rutinas como **abstracciones procedurales**:

Una abstracción procedural es una unidad que al ser invocada evalúa una instrucción y en general provoca un cambio en el estado de computación, esto es, modifica los valores del ambiente de referenciamiento de la llamada provocando un efecto colateral. El usuario de un procedimiento observa únicamente las transformaciones en el estado de la memoria pero no los pasos que dieron lugar a esta transformación. Las unidades definidas de esta manera son abstracciones a nivel instrucción y se invocan como cualquier otra instrucción.

✧ Rutinas como **abstracciones funcionales**:

Una abstracción funcional es una unidad que evalúa una expresión y al ser invocada produce un resultado. Idealmente no debería producir un efecto colateral.

Las unidades definidas de esta manera son una manera de hacer abstracción a nivel expresión. La manera de invocar a este tipo de rutinas es en el contexto de una expresión ya que tienen asociado un valor resultado.

En algunos lenguajes se hace el *return* del valor (el valor computado no se asigna a una variable), en otros se utiliza el nombre la función como área de almacenamiento. Así el nombre de la función tiene dos semánticas dependiendo de dónde aparezca:

1. Área de almacenamiento auxiliar (a la izquierda).
2. Invocación recursiva (a la derecha).

En lenguajes funcionales se evita la doble semántica sobre el nombre de la función.

La incorporación de las unidades implica que hay una cierta responsabilidad en la administración de memoria. Lo primero que hay que tener en cuenta es la **representación en memoria** de la unidad. Hay dos partes bien reconocibles:

- El **registro de activación** o stack frame.
- El **segmento de código** de la unidad.

El primero corresponde al espacio vinculado a los datos de la unidad. Estas dos partes no están habitualmente juntas en la memoria. Los segmentos de código se cargan en un lugar reservado para tal fin en la memoria, mientras que los registros de activación se guardan en otra porción de memoria. Es habitual que a pesar de que la memoria es única, las presentaciones hacen referencia a dos memorias, una identificada como D para hacer referencia la **memoria de datos**, una identificada como C para la **memoria de código**. La memoria D habitualmente se maneja como una pila. Más adelante aparecerá una tercera, llamada H, que es la **memoria de manejo libre**.

Como el registro de activación está asociado a los datos, vamos a notar que está dividido en dos partes:

|                                    |
|------------------------------------|
| <b>Objetos de dato del sistema</b> |
| <b>Objetos de datos</b>            |

Entre los **objetos de datos** se reserva espacio para los argumentos, valores de retorno (en caso de que los tenga), variables locales y temporales.

Los **objetos de datos del sistema** son datos que es necesario almacenar para garantizar el correcto funcionamiento de la implementación. Se guarda información como el puntero de retorno (dirección de la memoria de código donde se retoma la ejecución luego de que la unidad termina su ejecución). Además del puntero de retorno puede ser necesario guardar más información, la cual dependerá de la estructura del programa y el tipo de administración de memoria.

El acceso a cualquier objeto de dato dentro del registro de activación de una unidad se calcula como:

Dirección Base del Registro de Activación de la unidad + desplazamiento de la variable

Comenzando por la forma más limitada y avanzando hacia la más flexible, iremos presentando la evolución de registro de activación y las responsabilidades del traductor a la hora de procesar una invocación y una definición de una unidad.

Comencemos entonces con unidades sin parámetros y la administración de memoria más limitada que es la *estática*.

Un lenguaje con reglas de administración estática es generalmente compilado. Este tipo de administración no admite recursividad por lo que se sabe exactamente cuánta memoria es necesaria para ejecutar cualquier programa.

En compilación puede calcularse el alcance de cada variable y el ambiente de referenciamiento de cada unidad. También el espacio requerido para mantener cada variable local a la unidad y su desplazamiento dentro del registro de activación.

Se conoce el espacio que se requiere para la ejecución de todo el programa, es por eso que el único objeto de dato que hace falta es el puntero de retorno. No hace falta nada más porque en este tipo de lenguajes el ambiente global está declarado explícitamente. De esta manera, en la memoria de datos hay una sección destinada a las variables comunes o globales (es como un registro de activación más, solo que sin objetos de datos de sistema).

En la carga del programa, se reserva espacio de almacenamiento para todo el código y todos los registros de activación. Cada segmento de código queda asociado a un único registro de activación. También toman su valor definitivo el lugar donde está la primera instrucción de cada unidad en el área de código C y la dirección base de cada registro de activación en el área de datos. Estas dos direcciones se manejan con direcciones simbólicas durante la compilación cU y dU respectivamente, siendo U el nombre de la unidad.

Considerando un set de instrucciones podemos traducir la sección ejecutable de un programa. Veamos un ejemplo:

```
Subrutina P
integer i,j,k

i:= 1
k:= 2
j:= i + k
endSubrutina
```

La porción de código de la subrutina P, en un principio es la traducción de las instrucciones del lenguaje al comportamiento semántico equivalente escrito en el set de instrucciones de la máquina destino. En esta clase utilizaremos un set de instrucciones particular, para la práctica les presentaremos una versión del lenguaje *SimpleSem*. Vemos como podría verse el código entonces:

```
SET dP+1,1    // i:=1
SET dP+3,1    // k:=2
SET dP+2, D[dP+1]+D[dP+3] // j:=i+k
```

La instrucción SET guarda en la locación de memoria referenciada por la dirección que corresponde al primer argumento, el resultado de la expresión que recibe como segundo argumento. Como mencionamos anteriormente, los accesos a las variables se hacen desde la dirección base del registro de activación de la unidad con el correspondiente desplazamiento. Los compiladores suelen guardar esta información (los desplazamientos) en la tabla de símbolos para acelerar el proceso de traducción.

Si visualizamos el registro de activación, puede notarse el porqué de los desplazamientos que se observan en el código:

| Memoria de datos D |                    |    |
|--------------------|--------------------|----|
| 10                 | Puntero de retorno | dP |
| 11                 | i                  |    |
| 12                 | j                  |    |
| 13                 | k                  |    |

La numeración es anecdótica, en un manejo estático de memoria como el que estamos planteando, los registros de activación necesarios serán cargados en algún lugar de memoria que esté disponible en ese momento.

En este escenario lo único que hace falta para que todo funcione correctamente es guardar el puntero de retorno, ya que todo lo demás es conocido en compilación.

En la traducción del código, además de la traducción del código de la unidad, hay que prestar especial atención a lo que es necesario hacer al momento de la invocación de la unidad, en el prólogo (antes de la ejecución de la primera instrucción del bloque ejecutable) y el epílogo (luego de ejecutar la última instrucción del bloque ejecutable pero antes del retorno).

Las variables del entorno global están indicadas a través de la palabra *common*. Todas las variables que están declaradas de esta manera no tienen su espacio dentro de un registro de activación sino en una porción particular de memoria reservada para este tipo de variables (comunes a todas las unidades o globales). Esto es de esta manera en el contexto que estamos discutiendo, un lenguaje con asignación estática de memoria.

Veamos cómo se vería la memoria en un ejemplo completo.

```
Programa Principal
Common k
integer x
```

```
x:= 5
Call P
finPrograma
```

```
Subrutina P
integer i,j
```

```
i:= 1
k:= 2
j:= i + k
endSubrutina
```

| Memoria de código C |                              |                                                                           | Memoria de datos D |                 |
|---------------------|------------------------------|---------------------------------------------------------------------------|--------------------|-----------------|
| cPrincipal          | SET dPrincipal, 5            | <i>Guardar p. retorno</i><br><i>Invocación a P</i><br><i>Fin programa</i> | dCommon            | k               |
|                     | SET dP, IP+1                 |                                                                           | dPrincipal         | x               |
|                     | JUMP cP                      |                                                                           | dP                 | Puntero Retorno |
|                     | HALT                         |                                                                           |                    | i               |
| cP                  | SET dP+1, 1                  | <i>Epílogo de P</i>                                                       |                    | j               |
|                     | SET dCommon, 2               |                                                                           |                    |                 |
|                     | SET dP+2, D[dP+1]+D[dCommon] |                                                                           |                    |                 |
|                     | JUMP dP                      |                                                                           |                    |                 |

Noten que al guardar el puntero de retorno se guarda el valor del IP+1, el retorno debe ser a la primera instrucción luego de la invocación. En este tipo de administración de memoria la invocación requiere dos acciones: guardar el puntero de retorno y saltar al lugar donde comienza el código de la unidad llamada. De la misma manera el epílogo de la unidad solo requiere que el flujo de control siga por la instrucción indicada por el valor que guardamos como puntero de retorno.

Claramente este escenario no es el más frecuente en los lenguajes modernos, sin embargo nos es útil para ir entendiendo qué es lo que resulta necesario hacer, y dónde debemos hacerlo.

Modifiquemos el escenario pasando a un lenguaje con una administración de memoria [dinámica basada en pila](#).

En este nuevo esquema de administración de memoria, es posible determinar la distancia entre la unidad que referencia una variable y la unidad que contiene la declaración; pero ya no es posible anticipar el espacio de memoria requerido para ejecutar el programa. La responsabilidad de la administración de la memoria es asumida por el traductor, lo que implica que hay más código que debe generarse en la invocación y epílogo de las unidades. En ejecución se conoce la dirección base de cada registro de activación (pero no en carga como en el caso anterior); los registros de activación ocupan y liberan memoria siguiendo un comportamiento de pila y, al admitirse recursión, los segmentos de código pueden quedar asociados a varios registros de activación.

Veamos que hace falta:

1. Por un lado, el entorno global se resuelve siguiendo la estructura del programa. Así en un lenguaje con estructura de bloques chatos o con reglas de alcance dinámico es necesario conocer la identidad de la unidad llamadora. Esto se traduce en la necesidad de contar con un nuevo objeto de datos del sistema dentro del registro de activación de la unidad. Este objeto de dato nuevo se conoce como enlace dinámico.

En ese objeto de dato almacenaremos entonces la dirección base del registro de activación de la unidad llamadora.

✓ **IMPORTANTE:** En el caso de que el lenguaje tenga estructura de bloques anidados, el enlace dinámico no nos ayuda a identificar el ambiente global de manera adecuada. Es por eso que para este tipo de lenguajes hará falta un nuevo objeto de dato del sistema, el enlace estático. Este tipo de lenguajes son en general compilados y al tratarse de información que tiene que ver con la estructura estática del programa puede calcularse sin problemas en compilación. Por lo cual siempre se conoce el enlace estático a guardar en el momento que tenemos que hacerlo.

2. En el momento de la invocación es necesario:

- Reservar el espacio en la pila para el nuevo registro de activación.
- Guardar puntero de retorno.
- Guardar enlace dinámico.
- Guardar el enlace estático (de ser necesario).
- Identificar que el registro de activación en ejecución a partir de ahora será el recién creado.
- Actualizar IP con el comienzo del bloque ejecutable de la unidad llamada.

3. En el momento del epílogo de la unidad es necesario:

- Liberar la memoria correspondiente al registro de activación de la unidad que está terminando su ejecución.
- Actualizar la identidad del registro de activación que estará en ejecución a partir de la siguiente instrucción (para ello utilizamos el enlace dinámico).
- Actualizar IP utilizando el puntero de retorno, para que la ejecución continúe desde el punto adecuado del código del llamador.

Al tener que administrar la memoria, el sistema hace uso de dos registros rápidos: *actual* y *libre*. Ambos registros tienen almacenadas direcciones de la memoria de datos. *Actual* almacena el comienzo del registro de activación correspondiente a la unidad en ejecución, mientras que *libre* tiene la dirección de la primera locación "libre" o de la locación que sigue al fin del registro de activación marcado por actual. Esos registros van a reemplazar las etiquetas que teníamos antes para los registros de activación, esto se debe a que no sabemos en qué momento el registro está en la memoria y por lo tanto tampoco sabemos dónde. Para poder modificar estos registros nuestro set de instrucciones tiene la instrucción SETR que permite modificar el valor guardado en un registro rápido. La sintaxis de la misma es SETR registro, expresión.

Teniendo en cuenta lo que resulta necesario hacer, podemos ver ahora un ejemplo asumiendo que el lenguaje resuelve los accesos no locales por la cadena dinámica (es decir utilizando el enlace dinámico):

```
Programa Principal
```

```
integer i,k  
i:=3  
k:=5  
Call P  
finPrograma
```

```
Subrutina P
```

```
integer i,j  
i:=1  
j:= i + k  
Call Q  
endSubrutina
```

```
Subrutina Q
```

```
integer l  
l:= j *7
```

endSubrutina

Si miramos una foto de la memoria cuando se está ejecutando el código de Q tendríamos lo siguiente:

| Memoria de código C |                                                                      |                                                              | Memoria de datos D |                 |
|---------------------|----------------------------------------------------------------------|--------------------------------------------------------------|--------------------|-----------------|
| cPrincipa<br>l      | SETR <b>Libre</b> , <b>actual</b> +2                                 | Reserva espacio para las variables del programa principal    | 3                  | i               |
|                     | SET <b>actual</b> , 3                                                | Asignación a i                                               | 5                  | k               |
|                     | SET <b>actual</b> +1, 5                                              | Asignación a k                                               | Puntero Retorno    | Enlace dinámico |
|                     | SET <b>Libre</b> , IP+4                                              | Guardamos el P. Ret                                          | 1                  | i               |
|                     | SET <b>Libre</b> +1, <b>actual</b>                                   | Guardamos el ED                                              | 6                  | j               |
|                     | SETR <b>actual</b> , <b>Libre</b>                                    | Indicamos cual es el registro en ejecución a partir de ahora | Puntero Retorno    | <b>actual</b>   |
|                     | SETR <b>Libre</b> , <b>actual</b> +4                                 | 4 es el tamaño del RA de P                                   |                    | Enlace dinámico |
|                     | JUMP cP                                                              |                                                              |                    | l               |
|                     | HALT                                                                 | Fin programa                                                 |                    | <b>Libre</b>    |
|                     |                                                                      |                                                              |                    |                 |
| cP                  | SET <b>actual</b> +2, 1                                              |                                                              |                    |                 |
|                     | SET <b>actual</b> +3, D[ <b>actual</b> +2]+D[D[ <b>actual</b> +1]+1] | Ver comentario abajo (*)                                     |                    |                 |
|                     | SET <b>Libre</b> , IP+4                                              | Guardamos el P. Ret                                          |                    |                 |
|                     | SET <b>Libre</b> +1, <b>actual</b>                                   | Guardamos el ED                                              |                    |                 |
|                     | SETR <b>actual</b> , <b>Libre</b>                                    | Indicamos cual es el registro en ejecución a partir de ahora |                    |                 |
|                     | SETR <b>Libre</b> , <b>actual</b> +3                                 | 3 es el tamaño del RA de Q                                   |                    |                 |
|                     | JUMP cQ                                                              |                                                              |                    |                 |
|                     | SETR <b>Libre</b> , <b>actual</b>                                    | Liberamos la memoria correspondiente al RA de P              |                    |                 |
|                     | SETR <b>actual</b> , D[ <b>Libre</b> +1]                             | En actual ponemos el ED                                      |                    |                 |
|                     | JUMP D[ <b>Libre</b> ]                                               | Saltamos al puntero de retorno                               |                    |                 |
| cQ                  | SET <b>actual</b> +2, D[D[ <b>actual</b> +1]+3]*7                    | Asignación a la variable l                                   |                    |                 |
|                     | SETR <b>Libre</b> , <b>actual</b>                                    | Retorno de Q: liberar memoria                                |                    |                 |
|                     | SETR <b>actual</b> , D[ <b>Libre</b> +1]                             | Actualizamos el RA en actual                                 |                    |                 |
|                     | JUMP D[ <b>Libre</b> ]                                               | Saltamos al puntero de retorno                               |                    |                 |
|                     |                                                                      |                                                              |                    |                 |

(\*) SET **actual**+3, D[**actual**+2]+D[D[**actual**+1]+1]

Noten que la instrucción tiene varios accesos a memoria. Al hacer D[**actual**+2] está recuperando el valor almacenado en la variable local i. Al hacer D[D[**actual**+1]+1] está recuperando el valor de la variable k, como esta variable no es local para recuperarla hay que acceder a la base de quién la contiene, su enlace dinámico y desde ahí se hace el desplazamiento correspondiente. En caso de tener que retroceder más en la cadena dinámica se va accediendo al enlace dinámico tantas veces como haga falta. Así por ejemplo si tuviera que recuperar el valor de una variable z ubicada en un registro de activación a una distancia de dos en la cadena dinámica, el acceso se hubiese expresado de la siguiente manera:

D[D[D[**actual**+1]+1] + desplazamiento de z]

Con rojo está el acceso al enlace dinámico, una vez conseguido accedemos al enlace dinámico de este nuevo registro de activación, retrocedimos dos veces en la cadena dinámica. Estando ahora en la base del registro de activación que contiene a z, con su desplazamiento podemos conseguir finalmente su valor.

Pueden comprobar que la implementación es correcta, haciendo una simulación del código instrucción por instrucción. Van a ir notando como las variables toman valor, como los registros rápidos actual y libre van a ir cambiando el registro de activación y el lugar a partir del cual la memoria está libre. El registro actual comienza con la dirección base de lo que al momento de la carga se destina a la memoria de datos, la que en el apunte llamamos memoria D.



### ¿Cómo se calcula el enlace estático?

Supongamos que tenemos el siguiente ejemplo de estructura de bloques anidados:

```

Subrutina P
  Subrutina Q
    Subrutina R
      ....
    FinSubrutina
  ....
finSubrutina

Subrutina S
  ....
FinSubrutina
finSubrutina

```

Podemos identificar tres situaciones posibles:

- ✧ [Situación 1] Una unidad llama a otra que está en su mismo nivel léxico, en el caso del ejemplo podría darse cuando  $S \rightarrow Q$  o  $Q \rightarrow S$ .
- ✧ [Situación 2] Una unidad invoca a otra que está declarada dentro de ella. En el caso del ejemplo esta situación se da cuando Q invoca a R, es decir  $Q \rightarrow R$ . También cuando  $P \rightarrow Q$  o cuando  $P \rightarrow S$ .
- ✧ [Situación 3] Una unidad invoca a una unidad que la contiene. Nuevamente volviendo al ejemplo, esto sucedería cuando  $R \rightarrow Q$ , o bien cuando  $R \rightarrow P$  (pueden notar que en este caso la distancia es mayor).

Situación 1: En este caso, la distancia entre la unidad llamadora y la llamada es igual a cero (ambas tienen el mismo nivel léxico) por lo tanto en enlace estático de la unidad llamadora es el enlace estático de la unidad llamada también.

Si miramos el ejemplo y consideramos que en el código de Q hay una llamada a S, puede notarse que el registro de activación que es el enlace estático de Q también será el enlace estático de S. Luego la instrucción que debe agregarse para guardar el enlace estático en el registro de activación sería:

```
SET Libre+2, D[actual+2]
```

Recuerden que en Libre está el Puntero de Retorno, en Libre+1 el enlace dinámico, en Libre+2 estará el enlace estático y a partir de Libre+3 las variables locales si es que hay.

Situación 2: En este caso, la distancia entre la unidad llamadora y la llamada es igual a uno, por lo tanto el enlace estático de la unidad llamada es el registro de activación de la unidad llamadora. En el caso del ejemplo si en el código de Q hay una llamada a R, el enlace estático de R será el registro de activación de Q (que está señalado desde el registro rápido **actual**). Luego la instrucción que debe agregarse para guardar el enlace estático en el registro de activación sería:

```
SET Libre+2, actual
```

Situación 3: En este caso, la distancia entre la unidad llamadora y la llamada es negativa. Lo que implica que hay una instancia de la unidad que contiene a la actual suspendida anteriormente. Si miramos el ejemplo y tuviéramos una llamada a Q desde el código de R, lo que vemos es que la distancia entre ambas es -1, para recuperar el enlace estático para esta nueva activación de Q necesitamos recuperar el enlace estático de la activación anterior de Q que permitió que R llegara a la ejecución. En este caso como la distancia es -1 hay que retroceder una vez por la cadena estática, si la distancia fuera mayor habría que retroceder tantas veces como marca la distancia. Este caso podría darse en una llamada a P desde R, en el ejemplo. En esta situación la distancia sería -2, si accedemos al enlace estático de R recuperaríamos una instancia de Q, el

enlace estático de Q es una activación de P, por lo tanto el acceso al enlace estático del enlace estático de R nos permite conseguir la información que necesitamos.

Resumiendo, se debe recorrer la cadena estática tantos niveles como intervienen en la distancia.

La instrucción que debe agregarse para guardar el enlace estático en el registro de activación sería:

```
SET Libre+2, D[D[actual+2]+2] (si la distancia es -1)
```

```
SET Libre+2, D[D[D[actual+2]+2]+2] (si la distancia es -2)
```

Y así siguiendo

### Parámetros

La discusión que afrontamos hasta ahora es muy interesante, pero deja de lado los parámetros. Hay dos maneras de hacerle llegar los datos sobre los cuales computar a una unidad:

- ✓ A través del acceso a variables no locales visibles para la unidad.
- ✓ A través del pasaje de parámetros.

Los datos pasados como parámetros son accedidos a través de nombres que son locales a la unidad. El pasaje de parámetros es más flexible, ya que define computación parametrizada. El uso de variables no locales, entorno global o entorno implícito puede conducir a programas menos confiables.

### Terminología básica:

- ✧ **Argumento:** datos que forman la entrada de una unidad: datos no locales, parámetros y archivos externos (idealmente todos deberían ser parámetros).
- ✧ **Resultado:** variables no locales o archivos externos que también pueden modificarse además de los parámetros (se mantienen en el área de almacenamiento temporal, para las funciones no se retornan a través de los parámetros).
- ✧ **Parámetro formal:** dato particular dentro del área de referenciamiento de la unidad. Se declaran en el encabezado de la unidad, aclarando en general su tipo.
- ✧ **Parámetro actual:** variables, constantes, expresiones. Pueden ser locales, no locales o a su vez parámetros. La elección de qué se permite como parámetro actual afecta la implementación del lenguaje de programación.

### Correspondencia entre parámetros formales y actuales

Los lenguajes de programación pueden adoptar diferentes maneras de resolver la correspondencia entre parámetros formales y actuales.

- ✧ **Posicional:** La ligadura del parámetro actual con el formal se hace por posición. El primer parámetro actual se liga al primer parámetro formal y así sucesivamente. Es el más habitual (por simple y confiable).
- ✧ **Por nombre:** El nombre del parámetro formal al cual se quiere corresponder el parámetro actual aparece junto a este último. Su ventaja radica en que se independiza del orden, sobre todo cuando hay muchos parámetros. La desventaja radica en que el usuario de la rutina debe conocer los nombres de los parámetros formales. Se utiliza en general para librerías. Ada admite este tipo de correspondencia.

La correspondencia por nombre está vinculada con el concepto de inicialización implícita.

Así en la invocación no se incluye la lista completa sino solo la de aquellas variables que me interesa que tengan un valor diferente a las establecidas en la definición.

```
Procedure P(A,B:int:=1; C:real:=0)
```

```
Llamada: P(A=100)
```

Algunos lenguajes permiten correspondencia posicional y por nombre, pero no la combinación en una misma llamada. Otros lenguajes permiten posicional y por omisión. Esta elección genera muchos problemas si se combina con sobrecarga.

Llamada: `P(100)`

En llamadas como la anterior, la resolución se hace de manera posicional hasta que se agotan los parámetros actuales, el resto quedan definidos por los valores por defecto.

Una de las grandes preguntas que deben responder los lenguajes vinculadas a parámetros es: ***¿quiénes pueden ser parámetros?*** Para que el lenguaje no pierda ortogonalidad en este punto debería admitir datos y unidades como parámetros.

El problema es que para los lenguajes imperativos y orientados a objetos, el costo de permitir unidades como parámetros es muy alto. Más aún si no se realizan recortes con respecto a que tipo de unidades pueden ser parámetros. No profundizaremos los detalles aquí, pero para que tengan una idea es necesario tomar decisiones sobre si el lenguaje controlará el tipo de los parámetros y cuál será el ambiente de referenciamiento para el subprograma pasado como parámetro. La mayoría de los lenguajes del paradigma imperativo y orientado a objetos no permiten pasar unidades como parámetros, lo más cercano a lo que llegan es la opción de lenguajes como C y C++ en los que se admite pasar punteros a unidades, lo que claramente es una sobre-simplificación de lo que sería permitir la facilidad.

### Formas de pasajes de parámetros en el caso de datos

Si se trata de datos, el pasaje de parámetros puede ser:

- ✓ Referencia
- ✓ Por copia
  - ◆ Valor
  - ◆ Resultado
  - ◆ Valor-Resultado
- ✓ Por nombre → Vinculado a la evaluación de orden normal.
- ✓ Por necesidad → Vinculado a la evaluación perezosa.

El pasaje por referencia y por copia (en cualquiera de sus versiones) está vinculado a la evaluación ansiosa.

Este tipo de pasaje de parámetros utiliza espacio de memoria adicional para los parámetros formales. Se requiere entonces almacenamiento extra y el uso de operaciones de copia. **El parámetro formal en términos prácticos funciona como una variable local.**

**Por valor:** El parámetro formal se inicializa con el valor del parámetro actual en el momento de la llamada.

**Por resultado:** El parámetro actual recibe el valor del parámetro formal al terminar la ejecución normal de la unidad llamada.

**Por valor-resultado:** Es una combinación de los dos anteriores. El valor del parámetro actual es utilizado para inicializar el parámetro formal, el cual es tratado luego como una variable local. Cuando la unidad termina, se copia el valor del parámetro actual en el formal.

### ¿Qué sucede cuando implementamos estos pasajes de parámetros?

Veamos las alternativas para los pasajes de parámetros vinculados a la evaluación estricta. Para estos tipos de pasajes de parámetros, lo habitual es contar con una locación de memoria dentro de los registros de activación para cada parámetro formal.

En el caso de los pasajes de parámetros por copia, esta locación tiene el mismo comportamiento que el de una variable local, mientras que en el caso del pasaje de parámetros por referencia almacena una dirección de la memoria de datos, esa dirección es la que corresponde al objeto de dato pasado como parámetro actual.

Para el pasaje de parámetros por referencia, el parámetro actual debe ser una variable. En el caso de los parámetros por copia puede ser una expresión, aunque lo habitual es que en sus formas out (resultado) e in-out (valor-resultado) se restrinja a variables también. En caso de que el lenguaje no adopte esta postura, el resultado que saldría por los parámetros modo out se pierde.

La forma del registro de activación ahora es:

|                                    |
|------------------------------------|
| <b>Objetos de dato del sistema</b> |
| <b>Parámetros formales</b>         |
| <b>Variables Locales</b>           |

El tamaño del registro de activación se determina por los objetos de datos del sistema (PR+ED+EE o PR+ED), la cantidad de parámetros y la cantidad de variables locales. Recuerden que para cada uno de ellos se reservan tantas locaciones como sean necesarias para almacenar valores del tipo que tengan asociados.

La incorporación de los parámetros requiere tomar acciones adicionales en la invocación, prólogo y/o epílogo de la unidad dependiendo del pasaje de parámetros. Claramente hay que seguir haciendo todo lo que explicamos para una rutina sin parámetros.

En la presentación vamos a ir generando el código sobre el mismo ejemplo. Lo presentamos aquí y vamos a ir generando las distintas versiones a medida que avancemos.

```
Program Principal
Var x:T
```

```
Procedure Q(x:T)
x:= x-1
End q
```

```
Procedure P(y:T)
var z:T
```

```
y:=3
z:=-3
call Q(y)
call Q(z)
End p
```

```
x:= -11
call P(x)
End Principal
```

Los registros de activación de cada unidad se verían de esta manera:

| Registro Activación P | Registro Activación Q |
|-----------------------|-----------------------|
| Puntero de retorno    | Puntero de retorno    |
| Enlace dinámico       | Enlace dinámico       |
| Parámetro y           | Parámetro x           |
| Variable local z      |                       |

### Pasaje de parámetros por copia

#### Modo in: Pasaje por valor

Veamos el código necesario para administrar correctamente el pasaje de parámetros. Lo marcado con gris son las acciones que ya habíamos presentado para la traducción de una unidad sin parámetros.

En la **invocación**:

1. Guardar puntero de retorno.
2. Guardar enlace dinámico.
3. Guardar el enlace estático (de ser necesario).
4. Guardar el valor del parámetro actual en la locación que corresponde al parámetro formal. Esto se hace para cada uno de los parámetros.
5. Identificar que el registro de activación en ejecución a partir de ahora será el recién creado (actualizar el registro rápido actual).
6. Reservar el espacio en la pila para el nuevo registro de activación (actualizar el registro rápido libre teniendo en cuenta el tamaño del registro de activación).
7. Actualizar IP con el comienzo del bloque ejecutable de la unidad llamada.

En el **prólogo**: No hay acciones en esta implementación.

En el **epílogo**:

1. Liberar la memoria correspondiente al registro de activación de la unidad que está terminando su ejecución.
2. Actualizar la identidad del registro de activación que estará en ejecución a partir de la siguiente instrucción (para ello utilizamos el enlace dinámico).
3. Actualizar IP utilizando el puntero de retorno, para que la ejecución continúe desde el punto adecuado del código del llamador.

En el epílogo no se agrega ninguna acción por tratarse de una forma de pasaje de parámetros modo in.

La forma mostrada anteriormente es la utilizada por los lenguajes de programación, debido a que es una solución que requiere menos instrucciones y menos memoria, por lo cual es más eficiente. Si se pretende una pureza conceptual, la inicialización de los parámetros debería hacerse en el prólogo de la unidad en lugar de en la invocación. Para poder hacerlo así, hay que dejar en la pila fuera del registro de activación los valores para inicializar cada parámetro formal. Claramente para hacerlo así, se requieren el doble de locaciones de memoria y el doble de instrucciones como mínimo. La única ventaja es la pureza conceptual, es decir que cada unidad se haga cargo de sus parámetros.

Veamos cómo quedaría el código generado para el ejemplo propuesto:

| Memoria de código C |                                         |                                                           |
|---------------------|-----------------------------------------|-----------------------------------------------------------|
| cPrincipa<br>l      | SETR <b>Libre</b> , <b>actual</b> +1    | Reserva espacio para las variables del programa principal |
|                     | SET <b>actual</b> , -11                 | Asignación a i                                            |
|                     | SETR <b>Libre</b> , IP+5                | Guardamos el P. Ret                                       |
|                     | SET <b>Libre</b> +1, <b>actual</b>      | Guardamos el ED                                           |
|                     | SET <b>Libre</b> +2, D[ <b>actual</b> ] | Inicializar el parámetro formal con el valor del actual   |

|    |                                               |                                                              |
|----|-----------------------------------------------|--------------------------------------------------------------|
| cP | SETR <b>actual</b> , <b>Libre</b>             | Indicamos cual es el registro en ejecución a partir de ahora |
|    | SETR <b>Libre</b> , <b>actual</b> +4          | 4 es el tamaño del RA de P                                   |
|    | JUMP cP                                       |                                                              |
|    | HALT                                          | Fin programa                                                 |
|    | SET <b>actual</b> +2, 3                       |                                                              |
|    | SET <b>actual</b> +3, -3                      |                                                              |
|    | SETR <b>Libre</b> , IP+5                      | Guardamos el P. Retorno - Ver comentario abajo (*)           |
|    | SET <b>Libre</b> +1, <b>actual</b>            | Guardamos el ED                                              |
|    | SET <b>Libre</b> +2, D[ <b>actual</b> +2]     | Inicializar el parámetro formal con el valor del actual      |
|    | SETR <b>actual</b> , <b>Libre</b>             | Indicamos cual es el registro en ejecución a partir de ahora |
| cQ | SETR <b>Libre</b> , <b>actual</b> +3          | 3 es el tamaño del RA de Q                                   |
|    | JUMP cQ                                       |                                                              |
|    | SET <b>Libre</b> , IP+5                       |                                                              |
|    | SET <b>Libre</b> +1, <b>actual</b>            |                                                              |
|    | SET <b>Libre</b> +2, D[ <b>actual</b> +3]     |                                                              |
|    | SETR <b>actual</b> , <b>Libre</b>             |                                                              |
|    | SETR <b>Libre</b> , <b>actual</b> +3          |                                                              |
|    | JUMP cQ                                       |                                                              |
|    | SETR <b>Libre</b> , <b>actual</b>             | Liberamos la memoria correspondiente al RA de P              |
|    | SETR <b>actual</b> , D[ <b>Libre</b> +1]      | En actual ponemos el ED                                      |
| cQ | JUMP D[ <b>Libre</b> ]                        | Saltamos al puntero de retorno                               |
|    | SET <b>actual</b> +2, D[ <b>actual</b> +1] -1 | Asignación a la variable l                                   |
|    | SETR <b>Libre</b> , <b>actual</b>             | Retorno de Q: liberar memoria                                |
|    | SETR <b>actual</b> , D[ <b>Libre</b> +1]      | Actualizamos el RA en actual                                 |
|    | JUMP D[ <b>Libre</b> ]                        | Saltamos al puntero de retorno                               |

OBSERVACION IMPORTANTE: La cantidad de instrucciones que hay que sumarle al IP cuando guardamos el puntero de retorno va a ir cambiando en función de la cantidad de parámetros que tenga la unidad. Esta observación se aplica al resto de lo explicado en el documento.

### Modo out: Pasaje por resultado

Nuevamente ponemos en negro lo nuevo o diferente. Al tratarse de un modo de pasaje de parámetros en formato out, va ser necesario conocer la dirección del parámetro actual para poder volcar el resultado al finalizar la ejecución del bloque ejecutable de la unidad.

En la **invocación**:

1. Para cada parámetro actual guardar en la pila su dirección. Si se trata de una expresión se guarda una referencia a una locación anónima.
2. Guardar puntero de retorno.
3. Guardar enlace dinámico.
4. Guardar el enlace estático (de ser necesario).
5. Identificar que el registro de activación en ejecución a partir de ahora será el recién creado (actualizar el registro rápido actual).
6. Reservar el espacio en la pila para el nuevo registro de activación (actualizar el registro rápido libre teniendo en cuenta el tamaño del registro de activación).
7. Actualizar IP con el comienzo del bloque ejecutable de la unidad llamada.

En el **prólogo**: No hay acciones en esta implementación.

En el **epílogo**:

1. Copiar el valor del parámetro formal en la locación que corresponde al parámetro actual (con desplazamiento negativo se consigue recuperar la dirección correspondiente para tal fin).
2. Liberar la memoria correspondiente al registro de activación de la unidad que está terminando su ejecución.
3. Actualizar la identidad del registro de activación que estará en ejecución a partir de la siguiente instrucción (para ello utilizamos el enlace dinámico).

4. Actualizar IP utilizando el puntero de retorno, para que la ejecución continúe desde el punto adecuado del código del llamador.

Veamos como quedaría el código generado para el ejemplo propuesto:

| Memoria de código C |                                                |                                                                                                                      |
|---------------------|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| cPrincipa<br>l      | SETR <b>Libre</b> , <b>actual</b> +1           | Reserva espacio para las variables del programa principal                                                            |
|                     | SET <b>actual</b> , -11                        | Asignación a i                                                                                                       |
|                     | SETR <b>Libre</b> , <b>actual</b>              | Guardamos la dirección del parámetro actual                                                                          |
|                     | SETR <b>Libre</b> , <b>Libre</b> +1            | Dejamos libre en el lugar que corresponde al comienzo del RA de P                                                    |
|                     | SET <b>Libre</b> , IP+4                        | Guardamos el P. Ret                                                                                                  |
|                     | SET <b>Libre</b> +1, <b>actual</b>             | Guardamos el ED                                                                                                      |
|                     | SETR <b>actual</b> , <b>Libre</b>              | Indicamos cual es el registro en ejecución a partir de ahora                                                         |
|                     | SETR <b>Libre</b> , <b>actual</b> +4           | 4 es el tamaño del RA de P                                                                                           |
|                     | JUMP cP                                        |                                                                                                                      |
|                     | SETR <b>Libre</b> , <b>Libre</b> -1            | Liberamos las locaciones correspondientes a las direcciones de los parámetros actuales (este es el lugar de retorno) |
| cP                  | HALT                                           | Fin programa                                                                                                         |
|                     | SET <b>actual</b> +2, 3                        |                                                                                                                      |
|                     | SET <b>actual</b> +3, -3                       |                                                                                                                      |
|                     | SET <b>Libre</b> , <b>actual</b> +2            | Guardamos la dirección del parámetro actual                                                                          |
|                     | SETR <b>Libre</b> , <b>Libre</b> +1            | Dejamos libre en el lugar que corresponde al comienzo del RA de P                                                    |
|                     | SETR <b>Libre</b> , IP+4                       | Guardamos el P. Ret                                                                                                  |
|                     | SET <b>Libre</b> +1, <b>actual</b>             | Guardamos el ED                                                                                                      |
|                     | SETR <b>actual</b> , <b>Libre</b>              | Indicamos cual es el registro en ejecución a partir de ahora                                                         |
|                     | SETR <b>Libre</b> , <b>actual</b> +3           | 3 es el tamaño del RA de Q                                                                                           |
|                     | JUMP cQ                                        |                                                                                                                      |
|                     | SETR <b>Libre</b> , <b>Libre</b> -1            | Liberamos las locaciones correspondientes a las direcciones de los parámetros actuales (este es el lugar de retorno) |
|                     | SET <b>Libre</b> , <b>actual</b> +3            | Guardamos la dirección del parámetro actual                                                                          |
|                     | SETR <b>Libre</b> , <b>Libre</b> +1            | Dejamos libre en el lugar que corresponde al comienzo del RA de P                                                    |
|                     | SET <b>Libre</b> , IP+5                        |                                                                                                                      |
|                     | SET <b>Libre</b> +1, <b>actual</b>             |                                                                                                                      |
| cQ                  | SETR <b>actual</b> , <b>Libre</b>              |                                                                                                                      |
|                     | SETR <b>Libre</b> , <b>actual</b> +3           |                                                                                                                      |
|                     | JUMP cQ                                        |                                                                                                                      |
|                     | SETR <b>Libre</b> , <b>Libre</b> -1            | Liberamos las locaciones correspondientes a las direcciones de los parámetros actuales (este es el lugar de retorno) |
|                     | SET D[ <b>actual</b> -1], D[ <b>actual</b> +2] | Copiamos el valor del parámetro formal en el actual (comienzo epílogo)                                               |
|                     | SETR <b>Libre</b> , <b>actual</b>              | Liberamos la memoria correspondiente al RA de P                                                                      |
|                     | SETR <b>actual</b> , D[ <b>Libre</b> +1]       | En actual ponemos el ED                                                                                              |
|                     | JUMP D[ <b>Libre</b> ]                         | Saltamos al puntero de retorno                                                                                       |
|                     | SET <b>actual</b> +2, D[ <b>actual</b> +1]-1   | Asignación a la variable l                                                                                           |
|                     | SET D[ <b>actual</b> -1], D[ <b>actual</b> +2] | Copiamos el valor del parámetro formal en el actual                                                                  |
|                     | SETR <b>Libre</b> , <b>actual</b>              | Retorno de Q: liberar memoria                                                                                        |
|                     | SETR <b>actual</b> , D[ <b>Libre</b> +1]       | Actualizamos el RA en actual                                                                                         |
|                     | JUMP D[ <b>Libre</b> ]                         | Saltamos al puntero de retorno                                                                                       |

### Modo in-out: Pasaje por valor-resultado

Veamos el código necesario para administrar correctamente el pasaje de parámetros. Lo marcado con gris son las acciones que ya habíamos presentado para la traducción de una unidad sin parámetros.

En la **invocación**:

1. Guardar puntero de retorno.
2. Guardar enlace dinámico.
3. Guardar el enlace estático (de ser necesario).
4. Guardar el valor del parámetro actual en la locación que corresponde al parámetro formal. Esto se hace para cada uno de los parámetros.

- Identificar que el registro de activación en ejecución a partir de ahora será el recién creado (actualizar el registro rápido actual).
- Reservar el espacio en la pila para el nuevo registro de activación (actualizar el registro rápido libre teniendo en cuenta el tamaño del registro de activación).
- Actualizar IP con el comienzo del bloque ejecutable de la unidad llamada.

En el **prólogo**: Inicializar los parámetros formales con los valores de los parámetros actuales

En el **epílogo**:

- Copiar el valor del parámetro formal en la locación que corresponde al parámetro actual (con desplazamiento negativo se consigue recuperar la dirección correspondiente para tal fin)
- Liberar la memoria correspondiente al registro de activación de la unidad que está terminando su ejecución.
- Actualizar la identidad del registro de activación que estará en ejecución a partir de la siguiente instrucción (para ello utilizamos el enlace dinámico).
- Actualizar IP utilizando el puntero de retorno, para que la ejecución continúe desde el punto adecuado del código del llamador.

Veamos como quedaría el código generado para el ejemplo propuesto, puede notarse que es igual al anterior salvo por la inicialización de las locaciones correspondientes a los parámetros formales:

| Memoria de código C |                                                |                                                                                                                      |
|---------------------|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| cPrincipa<br>l      | SETR <b>Libre</b> , <b>actual</b> +1           | Reserva espacio para las variables del programa principal                                                            |
|                     | SET <b>actual</b> , -11                        | Asignación a i                                                                                                       |
|                     | SETR <b>Libre</b> , <b>actual</b>              | Guardamos la dirección del parámetro actual                                                                          |
|                     | SETR <b>Libre</b> , <b>Libre</b> +1            | Dejamos libre en el lugar que corresponde al comienzo del RA de P                                                    |
|                     | SET <b>Libre</b> , IP+4                        | Guardamos el P. Ret                                                                                                  |
|                     | SET <b>Libre</b> +1, <b>actual</b>             | Guardamos el ED                                                                                                      |
|                     | SETR <b>actual</b> , <b>Libre</b>              | Indicamos cual es el registro en ejecución a partir de ahora                                                         |
|                     | SETR <b>Libre</b> , <b>actual</b> +4           | 4 es el tamaño del RA de P                                                                                           |
|                     | JUMP cP                                        |                                                                                                                      |
|                     | SETR <b>Libre</b> , <b>Libre</b> -1            | Liberamos las locaciones correspondientes a las direcciones de los parámetros actuales (este es el lugar de retorno) |
| cP                  | HALT                                           | Fin programa                                                                                                         |
|                     | SET <b>Libre</b> +2, D[ <b>actual</b> -1]      | Inicialización del parámetro                                                                                         |
|                     | SET <b>actual</b> +2, 3                        |                                                                                                                      |
|                     | SET <b>actual</b> +3, -3                       |                                                                                                                      |
|                     | SET <b>Libre</b> , <b>actual</b> +2            | Guardamos la dirección del parámetro actual                                                                          |
|                     | SETR <b>Libre</b> , <b>Libre</b> +1            | Dejamos libre en el lugar que corresponde al comienzo del RA de P                                                    |
|                     | SETR <b>Libre</b> , IP+4                       | Guardamos el P. Ret                                                                                                  |
|                     | SET <b>Libre</b> +1, <b>actual</b>             | Guardamos el ED                                                                                                      |
|                     | SET <b>Libre</b> +2, D[ <b>actual</b> -1]      | Inicializar el parámetro formal con el valor del actual                                                              |
|                     | SETR <b>actual</b> , <b>Libre</b>              | Indicamos cual es el registro en ejecución a partir de ahora                                                         |
|                     | SETR <b>Libre</b> , <b>actual</b> +3           | 3 es el tamaño del RA de Q                                                                                           |
|                     | JUMP cQ                                        |                                                                                                                      |
|                     | SETR <b>Libre</b> , <b>Libre</b> -1            | Liberamos las locaciones correspondientes a las direcciones de los parámetros actuales (este es el lugar de retorno) |
|                     | SET <b>Libre</b> , <b>actual</b> +3            | Guardamos la dirección del parámetro actual                                                                          |
|                     | SETR <b>Libre</b> , <b>Libre</b> +1            | Dejamos libre en el lugar que corresponde al comienzo del RA de P                                                    |
|                     | SET <b>Libre</b> , IP+5                        |                                                                                                                      |
|                     | SET <b>Libre</b> +1, <b>actual</b>             |                                                                                                                      |
|                     | SET <b>Libre</b> +2, D[ <b>actual</b> -1]      | Inicializar el parámetro formal con el valor del actual                                                              |
|                     | SETR <b>actual</b> , <b>Libre</b>              |                                                                                                                      |
|                     | SETR <b>Libre</b> , <b>actual</b> +3           |                                                                                                                      |
|                     | JUMP cQ                                        |                                                                                                                      |
|                     | SETR <b>Libre</b> , <b>Libre</b> -1            | Liberamos las locaciones correspondientes a las direcciones de los parámetros actuales (este es el lugar de retorno) |
|                     | SET D[ <b>actual</b> -1], D[ <b>actual</b> +2] | Copiamos el valor del parámetro formal en el actual (comienzo epílogo)                                               |



cQ

|                                                |                                                         |
|------------------------------------------------|---------------------------------------------------------|
| SETR <b>Libre</b> , <b>actual</b>              | Liberamos la memoria correspondiente al RA de P         |
| SETR <b>actual</b> , D[ <b>Libre</b> +1]       | En actual ponemos el ED                                 |
| JUMP D[ <b>Libre</b> ]                         | Saltamos al puntero de retorno                          |
| SET <b>Libre</b> +2, D[ <b>actual</b> -1]      | Inicializar el parámetro formal con el valor del actual |
| SET <b>actual</b> +2, D[ <b>actual</b> +1]-1   | Asignación a la variable l                              |
| SET D[ <b>actual</b> -1], D[ <b>actual</b> +2] | Copiamos el valor del parámetro formal en el actual     |
| SETR <b>Libre</b> , <b>actual</b>              | Retorno de Q: liberar memoria                           |
| SETR <b>actual</b> , D[ <b>Libre</b> +1]       | Actualizamos el RA en actual                            |
| JUMP D[ <b>Libre</b> ]                         | Saltamos al puntero de retorno                          |

### Pasaje de parámetros por referencia

En ciertos aspectos este tipo de pasaje de parámetros resulta similar al pasaje de parámetros por valor, solo que en lugar de pasar un valor estamos pasando una dirección. Tiene las mismas salvedades con respecto a la implementación. Hay varias alternativas para realizarlo, pero la que los lenguajes terminan adoptando es la que se presenta a continuación (es la más barata en uso de espacio y cantidad de instrucciones).

En la **invocación**:

1. Guardar puntero de retorno.
2. Guardar enlace dinámico.
3. Guardar el enlace estático (de ser necesario).
4. Guardar la dirección del parámetro actual en la locación que corresponde al parámetro formal. Esto se hace para cada uno de los parámetros.
5. Identificar que el registro de activación en ejecución a partir de ahora será el recién creado (actualizar el registro rápido actual).
6. Reservar el espacio en la pila para el nuevo registro de activación (actualizar el registro rápido libre teniendo en cuenta el tamaño del registro de activación).
7. Actualizar IP con el comienzo del bloque ejecutable de la unidad llamada.

En el **prólogo**: No hay acciones en esta implementación.

En el **epílogo**:

1. Liberar la memoria correspondiente al registro de activación de la unidad que está terminando su ejecución.
2. Actualizar la identidad del registro de activación que estará en ejecución a partir de la siguiente instrucción (para ello utilizamos el enlace dinámico).
3. Actualizar IP utilizando el puntero de retorno, para que la ejecución continúe desde el punto adecuado del código del llamador.

En el epílogo no se agrega ninguna acción por tratarse de una forma de pasaje de parámetros ya que no es necesario volcar hacia afuera los resultados, las modificaciones se van haciendo durante la ejecución del bloque de la unidad.

Veamos como quedaría el código generado para el ejemplo propuesto:

| Memoria de código C |                                      |                                                              |
|---------------------|--------------------------------------|--------------------------------------------------------------|
| cPrincipa<br>l      | SETR <b>Libre</b> , <b>actual</b> +1 | Reserva espacio para las variables del programa principal    |
|                     | SET <b>actual</b> , -11              | Asignación a i                                               |
|                     | SETR <b>Libre</b> , IP+5             | Guardamos el P. Ret                                          |
|                     | SET <b>Libre</b> +1, <b>actual</b>   | Guardamos el ED                                              |
|                     | SET <b>Libre</b> +2, <b>actual</b>   | Inicializar el parámetro formal con la dirección del actual  |
|                     | SETR <b>actual</b> , <b>Libre</b>    | Indicamos cual es el registro en ejecución a partir de ahora |
|                     | SETR <b>Libre</b> , <b>actual</b> +4 | 4 es el tamaño del RA de P                                   |
|                     | JUMP cP                              |                                                              |
|                     | HALT                                 | Fin programa                                                 |

|    |                                   |                                                                                                                                                                    |
|----|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| cP | SET D[actual+2], 3                | En actual+2 hay una dirección, no es el lugar donde guardamos 3, por eso es necesario el acceso a memoria                                                          |
|    | SET actual+3, -3                  |                                                                                                                                                                    |
|    | SETR Libre, IP+5                  | Guardamos el P. Retorno - Ver comentario abajo (*)                                                                                                                 |
|    | SET Libre+1, actual               | Guardamos el ED                                                                                                                                                    |
|    | SET Libre+2, D[actual+2]          | Inicializar el parámetro formal con la dirección del actual, noten que al ser un parámetro por referencia, necesitamos guardar la dirección almacenada en actual+2 |
|    | SETR actual, Libre                | Indicamos cual es el registro en ejecución a partir de ahora                                                                                                       |
|    | SETR Libre, actual+3              | 3 es el tamaño del RA de Q                                                                                                                                         |
|    | JUMP cQ                           |                                                                                                                                                                    |
|    | SET Libre, IP+5                   |                                                                                                                                                                    |
|    | SET Libre+1, actual               |                                                                                                                                                                    |
|    | SET Libre+2, actual+3             | Inicializar el parámetro formal con la dirección del actual, en este caso como es una variable local es simplemente la dirección                                   |
|    | SETR actual, Libre                |                                                                                                                                                                    |
|    | SETR Libre, actual+3              |                                                                                                                                                                    |
|    | JUMP cQ                           |                                                                                                                                                                    |
|    | SETR Libre, actual                | Liberamos la memoria correspondiente al RA de P                                                                                                                    |
| cQ | SETR actual, D[Libre+1]           | En actual ponemos el ED                                                                                                                                            |
|    | JUMP D[Libre]                     | Saltamos al puntero de retorno                                                                                                                                     |
|    | SET D[actual+2], D[D[actual+1]]-1 | Asignación a la variable l: noten que el uso de los parámetros requieren un acceso más a memoria                                                                   |
|    | SETR Libre, actual                | Retorno de Q: liberar memoria                                                                                                                                      |
|    | SETR actual, D[Libre+1]           | Actualizamos el RA en actual                                                                                                                                       |
|    | JUMP D[Libre]                     | Saltamos al puntero de retorno                                                                                                                                     |

### Pasaje de parámetros por nombre

La técnica básica de implementación consiste en reemplazar cada referencia a un parámetro formal con una llamada a una rutina (*thunk*) que evalúa una referencia al parámetro actual en el entorno apropiado. Se crea una rutina para cada parámetro actual.

En cada referencia se llama a la rutina que calcula el valor del parámetro actual, así la sobrecarga en ejecución es importante y hay una pérdida de legibilidad interesante en la traducción.

En la pila se guarda la dirección de la primera instrucción de la rutina y ambiente de referenciamiento (actual generalmente).

Al momento del uso se hace una llamada a esta unidad sin parámetros que realiza el cálculo del parámetro actual. Estas rutinas habitualmente tienen pasaje de parámetros por resultado, pero al tratarse de abstracciones funcionales dejan el resultado en el tope de la pila o en un registro rápido para su utilización posterior.

### Pasaje de parámetros por necesidad

Requiere la misma estrategia que la implementación del pasaje por nombre, solo que la invocación de la unidad *thunk* que se definió o no se invoca nunca o se invoca una única vez. En caso de invocarse una vez, el valor se guarda en el espacio reservado para el parámetro formal dentro del registro de activación de la unidad, con el fin de reutilizar el valor en futuros usos del parámetro formal.